

Introduction

The purpose of this project is to explore various facets of Graphs and Graph Algorithms.

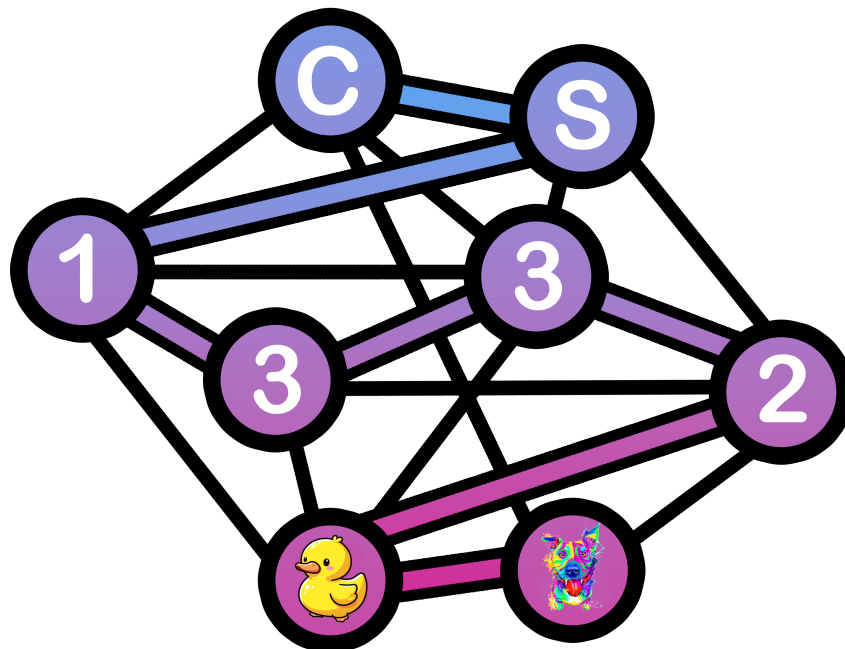
- **Refactor:** Learn how to implement Adjacency Lists, Disjoint Sets, and related methods to represent a mutable Graph.
- **Implement:** Learn how to write common Graph Algorithms as taught in lecture, translating them from concept to code: Depth-First Search, Breadth-First Search, Dijkstra's, Prim's, and Kruskal's.
- **Apply:** Learn how to utilize and adapt traditional algorithms to solve complex problems involving Graphs.

In this project, you will represent a Graph in code with an Adjacency List, and add methods to mutate the Graph (add and remove vertices and edges). Then, you will implement common Graph Algorithms as taught in class. Finally, you will utilize your mutable Graph class and Graph Algorithms to solve interesting problems and create functionality useful for city planning.

This is a complex assignment, so start early and ask many questions! Piazza is a great resource for viewing other students' questions and getting quick answers from TAs. Feel free to visit Office Hours for more direct help.

If you complete this project **by yourself**, you will learn a lot and build your competency in solving difficult problems with code, which will be extremely useful for future coursework, passion projects, and job interviews. Thus, we require that you do not use AI to ideate, debug, or write code, nor can you copy code from other students. Abusing AI will hurt your grade in the Demo and Exam. **AI usage of any kind is forbidden on this project. Violators will receive a zero and be referred to OSI.**

Note that for this project and unlike the previous projects, you should start with the Refactor portion, not Implement. This is because the Implement section will have you write algorithms for exploring and spanning graphs, which requires a functioning Graph structure first (unsurprisingly).



Provided Files

Main.java	The main class for your project. You will provide instantiation of your created instances, which is necessary to test your code.
MutableGraph.java	An abstract class representing a mutable Graph. You will extend this class.
StaticGraph.java	An interface for you to implement a simple, weighted, undirected, and mutable Graph with.
Vertex.java	A class to represent a Vertex in a Graph.
Edge.java	A class to represent an Edge in a Graph.
VertexDistance.java	A class to represent a Vertex and distance tuple in a Graph. This is useful for pairing together a Vertex and numerical distance/weight.
GraphAlgorithms.java	A shell class for you to implement the five Graph Algorithms in: DFS, BFS, Dijkstra's, Prim's, and Kruskal's.
DisjointSet.java	A class to represent a Disjoint Set.
DisjointSetNode.java	A class to represent a Disjoint Set node.
StaticWaddleWorks.java	An interface for you to implement WaddleWorks methods using your Graph and Graph Algorithms.
Intersection.java	A record class to represent an Intersection in a Graph of roads. Note the different Types.
Building.java	A record class to represent a Building in a Graph of electric wires.
MutableGraphStudentTests.java	Part 1 local testing file (non-comprehensive).
DisjointSetStudentTests.java	Part 1 local testing file (non-comprehensive).
GraphAlgorithmsStudentTests.java	Part 2 local testing file (non-comprehensive).
WaddleWorksStudentTests.java	Part 3 local testing file (non-comprehensive).

Deliverables

You will need to create your own files that implement the interfaces and abstract classes we provide. It does not matter what you name the files as long as you implement the required methods in `Main.java`.

You must submit **all** files that you create, along with your modified `Main.java`, `DisjointSet.java`, and `GraphAlgorithms.java`. To submit your code, follow the process below depending on your operating system:

Windows: With your project open in File Explorer, double click on `submit-windows.bat`.

MacOS/Unix: With your project open in your IntelliJ, open the integrated terminal (the `[>_]` icon). Run the following command: `./submit-unix.sh`.

These will create a file called `submission.zip` inside your project folder, and this is what you should upload to Gradescope.

Once submitted, double-check that it has uploaded properly on Gradescope. It will tell you if your code failed to compile.

Getting Started

1. **Download the files:** navigate to the Canvas assignment, and download and unzip the folder.
2. **Setup the project:** if you are using IntelliJ, open the project in the root directory of the unzipped project files. The 'src' directory should already be marked as sources root (blue) and the 'tst' directory should already be marked as test sources root (green). *Optional tip: right-click on the icon next to the top line of any Javadoc and select 'Render All Doc Comments' to view the Javadocs in a user-friendly format.*
3. **Complete part 1 first:** this is the **refactor** section. This part is a hard prerequisite to parts 2 and 3. Any classes you create in this part must be located in the respective package.
4. **Complete parts 2 and 3:** these are the **implement** and **apply** sections, respectively. Use the data structures you completed in part 1 to implement the algorithms in part 2 and create WaddleWorks in part 3. Any classes you create in these parts must be located in the respective packages.
5. **Test locally:** ensure you fill out the `Main.java` file with your instances from parts 1 and 3. From there, you can run the student test files located in `tst`. Note that these files are not comprehensive, and you are encouraged to write your own JUnits to test edge cases.
6. **Submit to Gradescope:** follow the above section's steps to zip your files and upload them to Gradescope, which will run the full suite of tests.

Part 1: Mutable Graph

Background

A Graph is a collection of vertices (nodes) and edges that connect the vertices. Think of it like a tree but without a hierarchy. Rather than having parents and children, vertices have incident (attached) edges and neighbors. Neighbors are vertices on the other end of the incident edge.

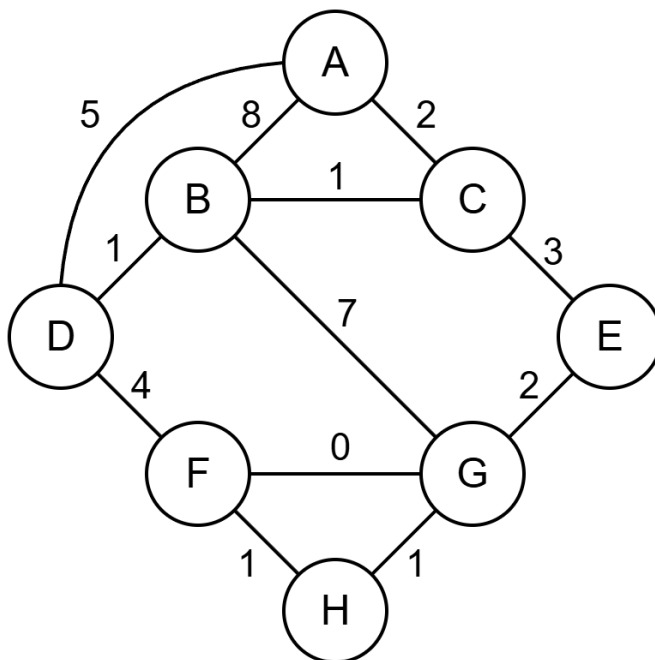
This project will involve exploring many aspects of graphs, including representing them in code, implementing basic algorithms using them, and building a real-world application with graph representations. Part 1 will focus on the first of these: representing graphs in code.

In addition to representing the graph, we want to be able to modify (mutate) it. This involves adding and removing vertices, as well as adding and removing edges.

Before considering the mutation aspect, let us actually build a graph. There are several ways to represent a graph in code. The two most common are an Adjacency List and an Adjacency Matrix. We will not utilize Adjacency Matrices for the purposes of this project, although there are cases where such a representation may be preferred, such as in a dense graph or in an application where your data has vertices enumerated from 0 to n .

Your graph should be Simple, Weighted, and Undirected (SWU). See the example below.

- Simple: There are no self-loops, and each pair of vertices has at most one edge between them.
- Weighted: The edges have numerical weights associated with them.
- Undirected: The edges are bidirectional. You can travel back and forth.



Adjacency List

One natural way to think of a graph is as a list of vertices. Furthermore, each vertex's relationship to the rest of the graph can be represented as a list of its connections. In other words, for every vertex, we could keep track of a list of its neighbor vertices and distances. Thus, when we access one vertex, we can easily see all of the other vertices that are immediately accessible from this one.

You should use a `HashMap` to back your Adjacency List, where the keys are all of the vertices in the graph, and the value for each key is that vertex's list of neighbors and the distance to those neighbors. Use a `VertexDistance` object to represent a vertex-distance pair. Note that all vertices should be present as keys in the Adjacency List, even if they have no neighbor vertices. Even if a vertex is isolated and alone, it is still an important part of the overall graph.

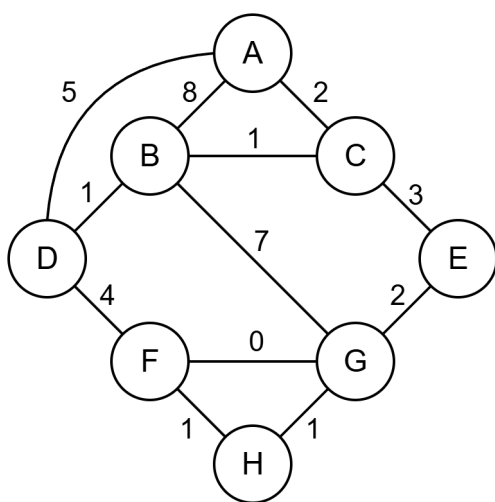
Keep in mind that you will have to update the Adjacency List any time you mutate the graph. For instance, if you remove a vertex, any other vertices that had a connection to the removed vertex must discard that information from each of their respective neighbor lists throughout the Adjacency List.

An important note is that, for the purposes of this project, your graph should be **UNDIRECTED**. This means that each edge is bidirectional. You can travel in either direction, from vertex A to B or from B to A. The `Edge` class considers $u \rightarrow v$ and $v \rightarrow u$ to be equivalent (see the `equals` method), so a set of Edges will *not* contain duplicated Edges for both directions since sets only contain unique objects.

The `MutableGraph` class, which you should extend, itself implements the `StaticGraph` interface, which requires providing the `getAdjList()` method. You should also consider what the constructor for your graph class should do as it relates to the Adjacency List.

Example

In your code, the `(vertex, distance)` tuples can be represented by a `VertexDistance` object. Note that since the edges are undirected, each edge is represented twice in the Adjacency List (for example, the association between A and C with weight 2 exists both in A's list and C's list).



$$\text{Adjacency List} = \left\{ \begin{array}{l} \text{A: } [(B,8), (C,2), (D,5)] \\ \text{B: } [(A,8), (C,1), (D,1), (G,7)] \\ \text{C: } [(A,2), (B,1), (E,3)] \\ \text{D: } [(A,5), (B,1), (F,4)] \\ \text{E: } [(C,3), (G,2)] \\ \text{F: } [(D,4), (G,0), (H,1)] \\ \text{G: } [(B,7), (E,2), (F,0), (H,1)] \\ \text{H: } [(F,1), (G,1)] \end{array} \right\}$$

StaticGraph vs. MutableGraph

Here is a brief overview of the two Graph classes provided in the project files.

- **StaticGraph**: a base interface for all graphs. This does *not* include methods that can modify the graph, but rather just the bare-bones edges, vertices, and adjacency information. This is information that is passed into graph algorithms, where you are not expected to mutate the graph in any way. Thus, this class is used when we need a graph that is not mutable, such as in Part 2: Graph Algorithms.
- **MutableGraph**: an abstract class that partially implements **StaticGraph**. **This is the class that you should extend.** We provide some starter code that handles the edges and vertices. You should extend this representation by implementing the abstract methods and the Adjacency List representation methods that remain unimplemented from **StaticGraph**. This graph implementation is used when we want a mutable graph, such as in Part 3: WaddleWorks.

Now, you should be able to create your own concrete Graph class and construct the Adjacency List.

Mutation Methods

In addition to representing the graph, we want to be able to modify (mutate) it. This involves adding and removing vertices, as well as adding and removing edges. Consider what would happen to a vertex's edges if we remove the vertex.

In a naive approach, we might just delete the vertex and leave the edges untouched. This would break the graph, because now we have edges that lead to nowhere, which violates an assumption underpinning graphs: edges connect two vertices. Thus, whatever you are using a graph to represent would lose its meaning, and graph algorithms would no longer work as expected.

This gives us one thing to consider when mutating a graph: you should handle hanging edges when deleting a vertex.

The methods you must implement that mutate the graph are as follows:

1. `addVertex(Vertex<T> vertex)` → Add a vertex to the graph. Note that it defaults to having no connections (edges) to other vertices. You should take a look inside the **Vertex** class for context.
2. `addEdge(Edge<T> edge)` → Add an edge to the graph, **including its endpoints**. Note that an **Edge** object contains its endpoint vertices as attributes. You should take a look inside the **Edge** class for context.
3. `removeVertex(Vertex<T> vertex)` → Remove a vertex from the graph, including its incident edges.
4. `removeEdge(Edge<T> edge)` → Remove an edge from the graph.

For all of these methods, consider what attributes of the class you would want to modify, as well as any implications for the Adjacency List. **The Adjacency List should always represent all of the graph's vertices and connections, even for vertices with no edges.**

See the "Exception Handling" section below for how to handle edge cases.

Other Methods

All methods that you should include in your graph class are listed as abstract methods in `MutableGraph`, which you must extend. Here is a brief summary of each of those not discussed in the previous sections:

1. *Constructor* $\rightarrow$ Instantiate a graph with a set of vertices, set of edges, and Adjacency List. Verify that the passed-in edge and vertex sets are consistent and valid (see exception handling below).
2. `getAdjList()` $\rightarrow$ Return the graph's Adjacency List in $O(1)$.
3. `getNeighbors(Vertex<T> vertex)` $\rightarrow$ Return a list of a vertex's neighbors and their edge weights in $O(1)$. Very useful for graph algorithms.

There are also a handful of methods implemented for you in `MutableGraph` or `StaticGraph` that are useful for you to use in your graph class and elsewhere.

1. `getVertexCount()` $\rightarrow$ Returns the number of vertices in the graph.
2. `getEdgeCount()` $\rightarrow$ Returns the number of edges in the graph.
3. `containsVertex(Vertex<T> vertex)` $\rightarrow$ Returns true if the graph contains the vertex.
4. `containsEdge(Edge<T> edge)` $\rightarrow$ Returns true if the graph contains the edge.

Exception Handling

Many of these methods require some exception handling. We recommend you brainstorm all of the different ways the inputs must be validated in order for the methods to function correctly, and handle them gracefully.

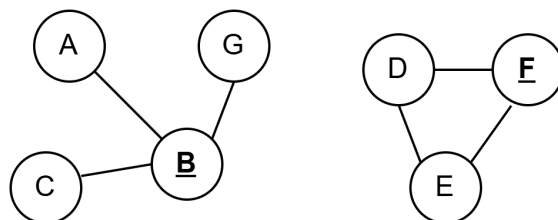
Here is a list of situations to throw an `IllegalArgumentException` for:

- In the constructor, the vertex set used to construct the graph must contain all of the vertices referenced in the edge set.
- The vertex set and edge set used to construct the graph must not be null.
- The vertex or edge to be added must not be null.
- The vertex or edge to be added must not already exist in the graph.
- The vertex or edge to be removed must exist in the graph.
- The vertex to fetch neighbors for must be non-null and exist in the graph.
- The vertex cannot have a self-looping edge.

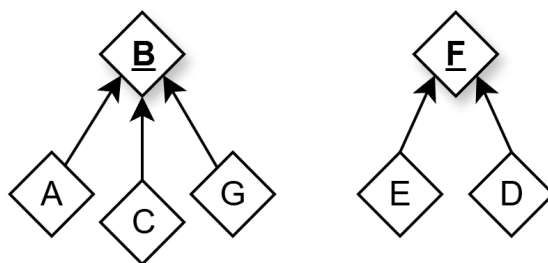
Now, you should be able to implement the rest of your Graph class!

Disjoint Set

A Disjoint Set (or Union-Find) data structure is a way of tracking which elements belong to different groups. Disjoint Sets are useful for tracking **connected components** of a graph. A connected component is a subset of the graph where every vertex in the subset can reach every other vertex in that same subset. Two disconnected clusters would be two different (*disjoint*) sets.



A disconnected graph composed of two connected components.



A Disjoint Set representing the above graph, with B and F as the roots.

A good way to tell which graph component a particular element belongs to is by using a Disjoint Set, as illustrated above.

First, take a look at the `DisjointSet` and `DisjointSetNode` classes to familiarize yourself with this data structure. We have provided most of the code to you for this, but you will need to modify `DisjointSet` to add some more functionality.

You will implement `getRoots`, which should return the set of all of the roots that constitute the whole Disjoint Set. A root can be considered the representative vertex in a cluster of vertices, and different disjoint clusters of vertices will thus each have its own root vertex. In a Disjoint Set with 5 separate connected components, there should be 5 roots. In that scenario, this method should return those 5 roots. In the diagram above, the returned set would be `{B, F}`.

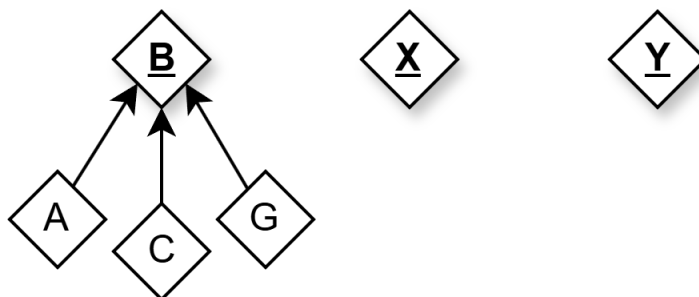
To implement `getRoots` efficiently, you should modify the provided Disjoint Set methods to dynamically keep track of the current state of the Disjoint Set in a backing structure. Refer to the Javadocs for more information. **Note that `getRoots` must be efficient and run in $O(1)$ time.**

See the Disjoint Set examples on the next page. For more in-depth information, see the **Kruskal's Algorithm: Disjoint Sets** page on the Canvas Modules. You can also look at [CSVistool.com](https://www.cs.vt.edu/~cs1332/CSVistool.com).

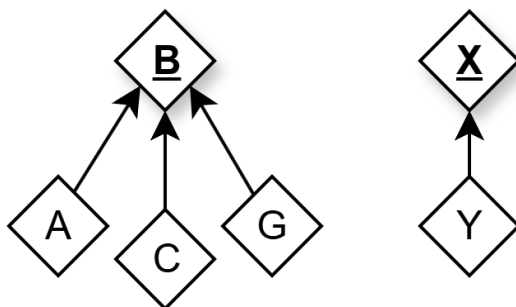
Disjoint Set Methods

The `find` method returns the root element of the set to which the passed-in element belongs, and additionally adds a new node to the Disjoint Set if the passed-in element is nowhere in the structure. By default, it has no relation to any other nodes, so it will become the root of its own brand-new set.

For example, say we already have the existing set with root B . Now, we call `find(X)` and `find(Y)`:



The `union` method merges two sets together, and one of the roots will be chosen as the overall root for the combined set. This is determined by using *rank*, which you can explore in the code. The first parameter is chosen if both ranks are the same. Say we call `union(X, Y)`:

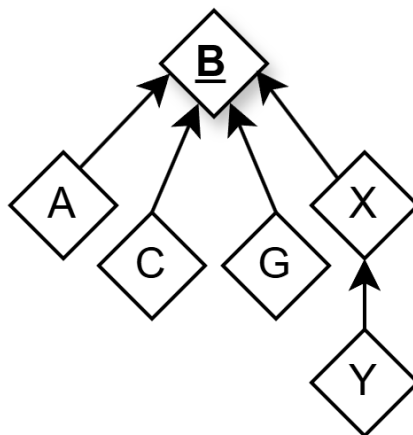


Now there are only two sets, with roots B and X . The user may want to use `find` to determine if two elements are in the same set or not. Consider the following outputs, and think about how you can use the output to make that determination.

1. `find(A)` $\rightarrow B$
2. `find(B)` $\rightarrow B$
3. `find(C)` $\rightarrow B$
4. `find(G)` $\rightarrow B$
5. `find(X)` $\rightarrow X$
6. `find(Y)` $\rightarrow X$

How would you determine if, say, B and Y are in the same set?

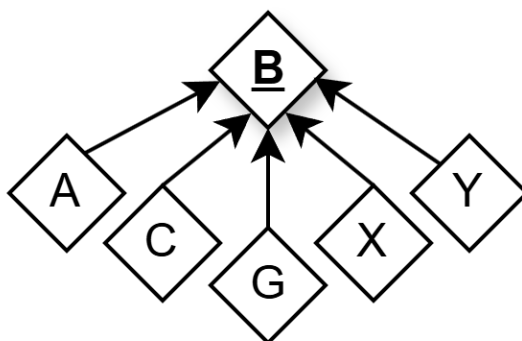
Finally, we will combine the two sets into one. To demonstrate that you can call `union` on any element, not just the root, let us call `union(A, Y)`.



The `union` method will find the root of *A*, which is *B*, and similarly find the root of *Y*, which is *X*. Then, it will choose whichever root has the higher rank to make the overall root for the entire combined set; in this case, *B* is chosen. Thus, *X* now points to its parent *B*.

X is no longer a root, so it should not be returned in `getRoots`, and all of the elements contained in its set are now part of the merged set. Calling `find` on any element above will return *B*.

As the user continues calling `find`, the disjoint set performs “path compression” to improve the efficiency of lookups. For example, *Y* will change its parent to *B* instead of being a child of *X*:



In the code, note that there are several overloaded methods for `find` and `union` that serve as private helper methods for the code.

Part 2: Graph Algorithms

Background

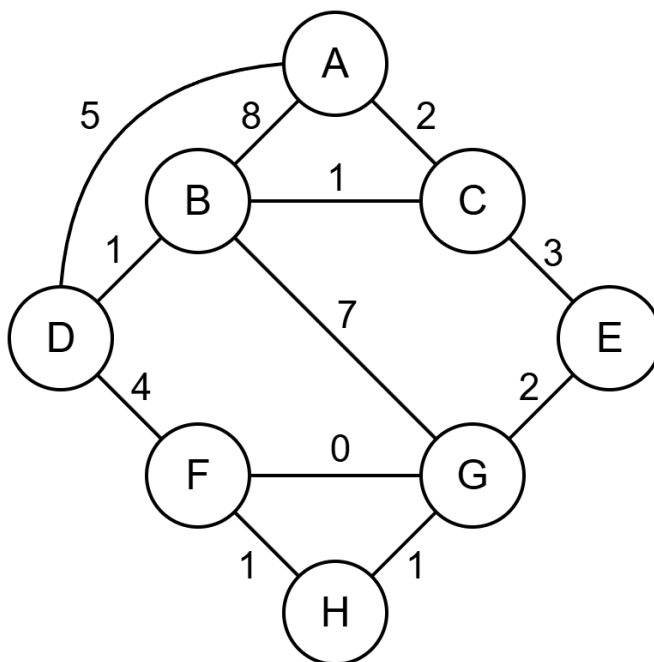
For Part 2, you will implement five world-famous Graph Algorithms as taught in lecture and in the module videos. If you need a refresher for any of them, refer to the module videos. We also provide a short explanation for each here.

First, note the different objects you will need to use for this section. We recommend inspecting these classes before you begin so you know what you have to work with.

- **StaticGraph**: The Graph interface that all algorithms will use as an input parameter.
- **Vertex**: An object to represent a Vertex with data in a graph (like a node).
- **VertexDistance**: A “pair” object that couples together a Vertex and an integer distance value.
- **Edge**: An object to represent an Edge with an integer weight and two Vertex endpoints u and v .

Use [CSVistool.com](https://www.csvistool.com) to visualize how these algorithms should behave. Pseudo-code exists for all of them in case you get stuck, but we recommend first giving it a good attempt yourself based on your conceptual understanding.

For all examples described below, refer to the following graph:



2.1 Breadth-First Search

Breadth-First Search (BFS) explores a connected graph in layers from a start vertex until all accessible vertices have been visited, and outputs a list of vertices *in the order they were visited*. BFS *does not* consider edge weights.

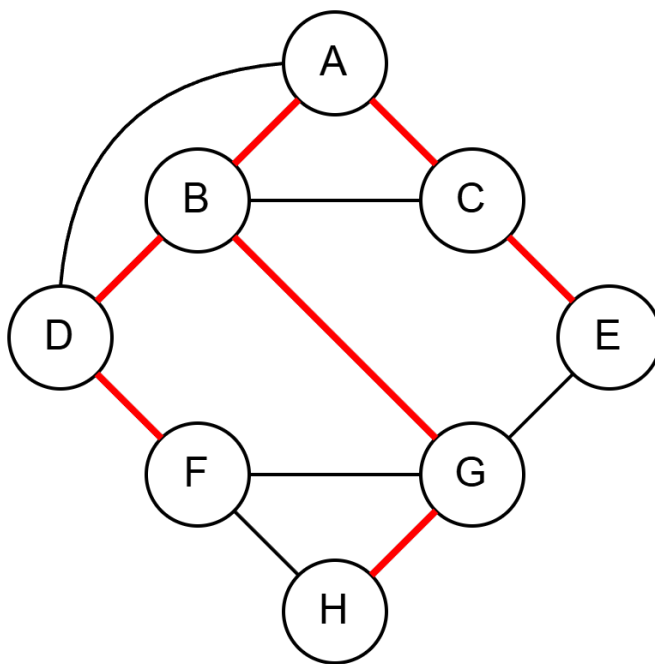
In the case of there being multiple distinct connected components in a graph, BFS terminates when it finishes exploring the component in which the start vertex is contained, and it will not visit any other components of the graph.

BFS explores all vertices that are one edge away from the start vertex, then two edges away, then three, and so on. This is useful for finding the shortest path by number of edges (but not by edge weights).

Some edge cases to consider throwing an `IllegalArgumentException` for include the start vertex being undefined or not existing in the graph, or the graph itself being undefined.

Example

Consider the graph on the previous page. Begin BFS by starting at **Vertex A**. BFS explores the graph by visiting the all neighbors of the current vertex, breaking ties via lexicographic order. The output is the list of vertices in the order they were visited.



Output = [A, B, C, D, G, E, F, H]

2.2 Depth-First Search

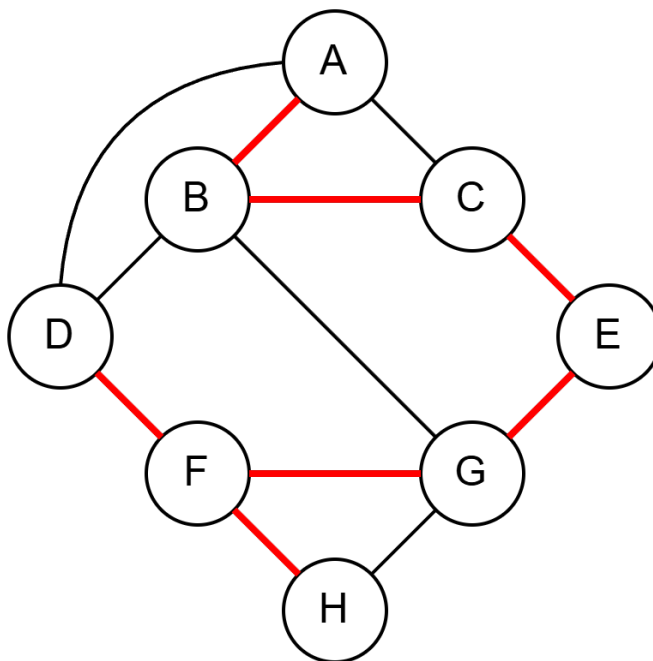
Depth-First Search (DFS) explores a connected graph naively from a start vertex until all accessible vertices have been visited, and outputs a list of vertices *in the order they were visited*. DFS *does not* consider edge weights.

In the case of there being multiple distinct connected components in a graph, DFS terminates when it finishes exploring the component in which the start vertex is contained, and it will not visit any other components of the graph.

Some edge cases to consider throwing an `IllegalArgumentException` for include the start vertex being undefined or not existing in the graph, or the graph itself being undefined.

Example

Begin DFS by starting at **Vertex A**. DFS explores the graph by recursively visiting an adjacent vertex to the current vertex, breaking ties via lexicographic order. The output is the list of vertices in the order they were visited.



Output = [A, B, C, E, G, F, D, H]

2.3 Dijkstra's Shortest Path

Dijkstra's algorithm explores a connected graph from a start vertex until all accessible vertices have been visited, and outputs a Distance Map containing all of the vertices reachable from the start vertex and their corresponding shortest distances. The shortest distance between two vertices u and v , for the purposes of Dijkstra's, is defined as the sum of all edges traversed from u to v in the path with the lowest cumulative weight.

In the case of there being multiple distinct connected components in a graph, Dijkstra's should terminate when it finishes exploring the component in which the start vertex is contained, and it will not visit any other components of the graph. Unvisited vertices should have a distance of *infinity* (`Integer.MAX_VALUE`) in the Distance Map, since they are unreachable from the start vertex.

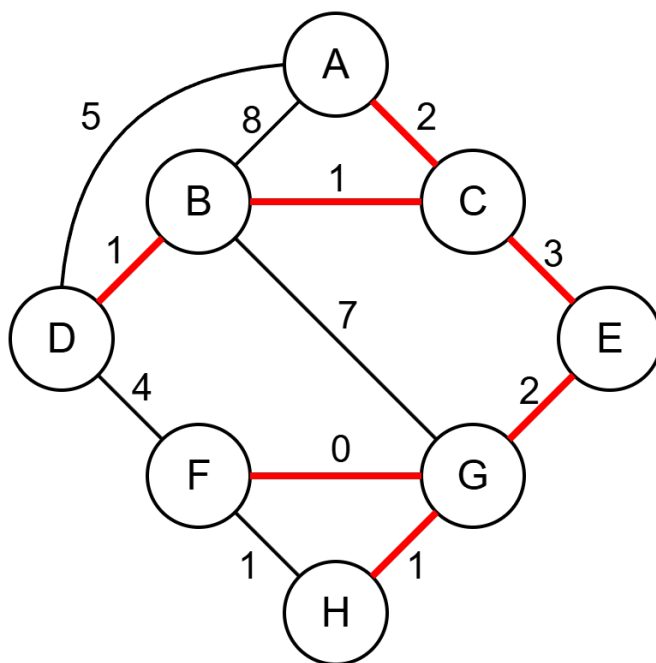
Dijkstra's explores vertices greedily by choosing the lowest cost path (the edge which would result in the lowest cumulative weight from the start vertex) accessible from the current visited set. This can be achieved with a **Priority Queue**.

Some edge cases to consider throwing an `IllegalArgumentException` for include the start vertex being undefined or not existing in the graph, or the graph itself being undefined.

Example

Begin Dijkstra's by starting at **Vertex A**. Dijkstra's explores the graph by considering all of the neighbors of the current vertex, and visiting the one accessed by the lowest weighted path, breaking ties via lexicographic order. When determining where to go next, Dijkstra's should consider the *cumulative* distance from the start vertex to this current vertex, not just the next edge's weight itself.

The output is the Distance Map of each vertex and its shortest total distance from Vertex A.



$$\text{Output} = \left\{ \begin{array}{l} \text{A: } 0 \\ \text{B: } 3 \\ \text{C: } 2 \\ \text{D: } 4 \\ \text{E: } 5 \\ \text{F: } 7 \\ \text{G: } 7 \\ \text{H: } 8 \end{array} \right\}$$

2.4 Prim's Minimum Spanning Tree

Prim's algorithm explores a connected graph from a start vertex until all accessible vertices have been visited, and outputs a Minimum Spanning Tree of edges connecting all of the vertices.

To recap, a Spanning Tree is a set of $|V| - 1$ edges that connect all vertices in the graph. Because it is a tree, there are no loops or cycles. A **Minimum** Spanning Tree has the additional requirement of minimizing the cumulative weight of its edges. In other words, out of all of the possible Spanning Trees for a graph, the MST is the one which has the lowest total edge weight. This is a useful concept when trying to design a system with the minimum number of connections and minimum costs, like electric wires connecting buildings.

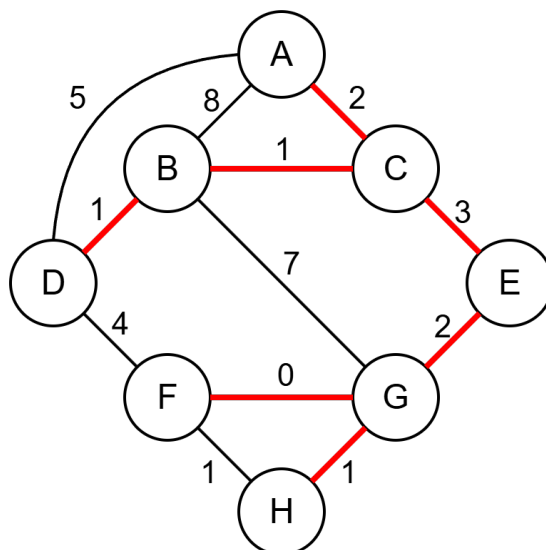
Prim's explores vertices greedily by choosing the lowest cost edge accessible from the current visited set. This can be achieved with a **Priority Queue**. Unlike Dijkstra's, we do not consider the cumulative path weight here; we care about individual edge weights.

Some edge cases to consider throwing an `IllegalArgumentException` for include the start vertex being undefined or not existing in the graph, or the graph itself being undefined.

Additionally, in the case of there being multiple distinct connected components in a graph, Prim's should return `null` since no MST exists. It is not possible to create an MST for a graph if some vertices are inaccessible. See *Minimum Spanning Forests* if you are curious about how this problem can be approached.

Example

Begin Prim's by starting at **Vertex A**. Prim's explores the graph by considering all of the neighbors of the current vertex, and visiting the one accessed by the lowest-weighted edge, breaking ties via lexicographic order. The output is the Set of $|V| - 1$ edges constituting a MST, or `null` if no MST exists. Note that there can be multiple equally-valid MSTs in a graph (for example, consider if all edge weights are equal).



MST = {AC, CB, BD, CE, EG, GF, GH}

2.5 Kruskal's Minimum Spanning Tree

Kruskal's algorithm greedily chooses the lowest-weight edge from a graph and adds it to the MST as long as it does not create a cycle.

This can be implemented by treating the initial set of edges as a **Priority Queue** and repeatedly popping the lowest edge not yet considered. Kruskal's discards edges that would create a cycle in the current MST.

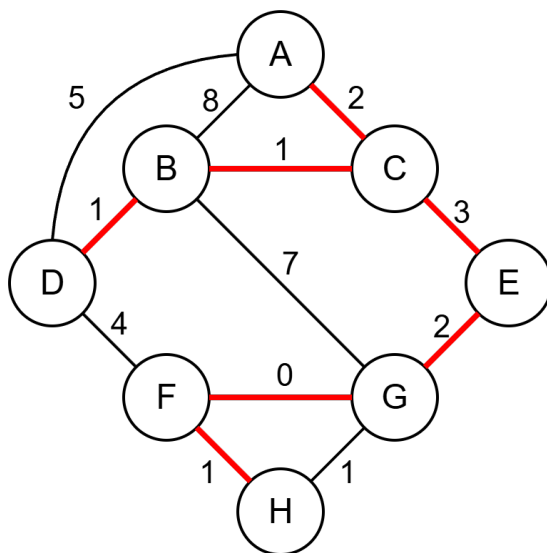
Kruskal's determines if adding an edge would create a cycle by using a **Disjoint Set**. Union subsets of vertices connected by edges in the MST being built, and use Find to check if a new edge being considered has both of its vertices in the same set already. If so, adding this edge would create a cycle.

An edge case to consider throwing an `IllegalArgumentException` for is if the graph is undefined.

Additionally, in the case of there being multiple distinct connected components in a graph, Kruskal's should return `null` since no MST exists. It is not possible to create an MST for a graph if some vertices are inaccessible. See *Minimum Spanning Forests* if you are curious about how this problem can be approached.

Example

Begin Kruskal's by adding all edges into a Priority Queue. Repeatedly pop the next lowest weight edge, check if it would create a cycle, and add it to the MST if not. The output is the Set of $|V| - 1$ edges constituting a MST, or `null` if no MST exists. Note that there can be multiple equally-valid MSTs in a graph (for example, consider if all edge weights are equal).



$\text{MST} = \{\text{FG}, \text{BC}, \text{BD}, \text{FH}, \text{AC}, \text{EG}, \text{CE}\}$

Part 3: WaddleWorks

Background

You've been hired to help implement parts of a city services application: WaddleWorks! More specifically, your expertise in Graphs is needed to solve some challenges related to **road infrastructure** and the **electrical grid**.

You will need to use the data structures and algorithms you created in Parts 1 and 2. Be mindful of any efficiency requirements listed in each method.

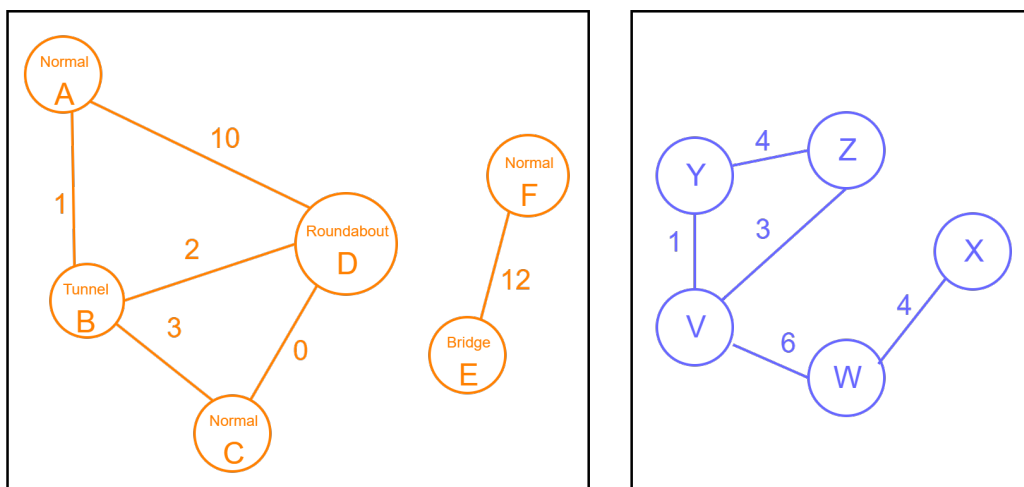
To begin, read through `StaticWaddleWorks.java` to get an overview of this section, and then create a class to implement the interface.

For the purposes of this section, we will often refer to two distinct (but related) graphs, the **Road** graph and the electrical **Grid** graph. The former uses **Intersections** as vertices, and the latter uses **Buildings** as vertices. These record objects are defined in their respective class files.

Intersections are connected by **roads**, with **time duration** (how long it takes to travel on the road) as the edge weight. Buildings are connected by **wires**, with **length** as the edge weight. For both, you can assume the sum of all edge weights in the graph will not exceed `Integer.MAX_VALUE`.

Here are some properties to keep in mind for these graphs:

- **Road Graph:** We can only *add* roads, not remove them. Once two Intersections are connected, they can never become disconnected. In WaddleWorks, a connected component of Intersections can be considered a “neighborhood,” and the Road graph can have any number of neighborhoods.
- **Grid Graph:** The graph must be fully connected; that is, each Building must be accessible from every other Building. Additionally, Buildings can store their closest Intersection as an attribute, which must always exist in the Road graph if defined.



See the above example graphs. The Road graph (orange) has different types of Intersections as its vertices, and the edges are roads with a time duration. Note that there are two neighborhoods in the Road graph ($\{A, B, C, D\}$, $\{E, F\}$).

The Grid graph (blue) is fully connected, and its vertices are Buildings. The edges are wires with a numerical length.

Part 3a: Standard Methods

For the methods in this part, the following terms will be used for efficiency purposes:

- $|I| \rightarrow$ number of Intersections in the Roads Graph
- $|R| \rightarrow$ number of Roads in the Roads Graph
- $|B| \rightarrow$ number of Buildings in the Grid Graph
- $|W| \rightarrow$ number of Wires in the Grid Graph

1. Constructor

To instantiate your WaddleWorks instance, you should take in parameters of type `MutableGraph` for an initial Road graph and an initial Grid graph. If either of these inputs are invalid or violate a constraint described on the previous page, throw an `IllegalArgumentException`.

The constructor is a good place to set-up your WaddleWorks program with instance variables and/or data structures to make your life easier. As you read through the methods below, consider what additional structures or variables could be useful to instantiate within this constructor.

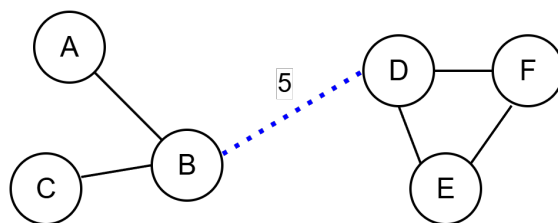
The constructor is very important. If the constructor has any bugs, your code will not pass most or any tests. Additionally, it should run in linear time (that is, $O(|I| + |R| + |B| + |W|)$).

2. addRoad

This method creates a road of a given duration between two Intersections in $O(1)$ time. This should add an edge in your Road graph, and new Intersections as needed. In the default case, this method should return a boolean value of `false`. However, if this newly-added road connects two previously disjoint neighborhoods, return `true`.

A WaddleWorks “neighborhood” is a connected component of Intersections. One Intersection in a neighborhood is accessible from all other Intersections in the same neighborhood, but *not* from any Intersections in other neighborhoods.

Throw an `IllegalArgumentException` if the duration is negative (time can’t be negative!), or if either Intersection is null. **Note: a newly-added Intersection does *not* count as its own neighborhood for the purpose of this method’s return value.**



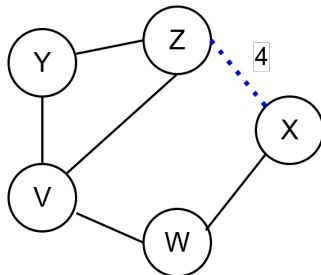
There are two previously-distinct connected components of vertices here. The first consists of Intersections $\{A, B, C\}$ while the second has Intersections $\{D, E, F\}$.

Calling `addRoad(B, D, 5)` creates a new road between Intersections B and D with distance 5, and since it connects two separate neighborhoods, it returns `True`.

3. addWire

This method creates a wire of a given length between two Buildings in $O(1)$ time. This should add an edge in your electrical Grid graph, adding new Buildings as necessary. Note that the Grid graph must always be fully connected.

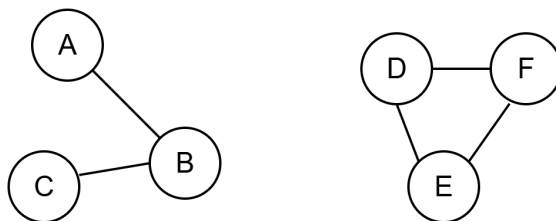
Throw an `IllegalArgumentException` if the length is negative, if either Building is null, or if this operation would cause the Grid graph to have distinct connected components. You must also validate that for Buildings with a non-null closest Intersection, the Intersection is already present in the Road graph.



Calling `addWire(Z, X, 4)` creates a new wire between Buildings *Z* and *X* with length 4.

4. getNeighborhoodCount

This method returns the number of neighborhoods in the Road graph in $O(1)$ time. Remember that a neighborhood (in WaddleWorks) is a connected component of Intersection vertices, and therefore the number of neighborhoods is just the number of connected components in the Road graph.



Calling `getNeighborhoodCount()` returns 2 in this example, as there are two distinct connected components made up of Intersection vertices on the Road graph.

5. Getters

Implement the methods `getRoads()` and `getGrid()` to return the current state of the Road graph and electrical Grid graph, respectively, in $O(1)$ time. These should be *very simple* to implement if you structured your WaddleWorks class well!

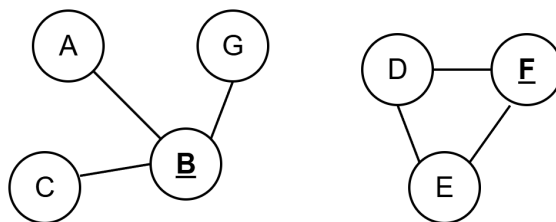
Part 3b: Challenge Methods

6. getNeighborhoodsByConnections

This method returns a Map of all neighborhoods and with set of their Intersections that have a number of connecting roads within the given *inclusive* range. The number of roads connected to an Intersection is the *degree* of the vertex.

The Map keys are the neighborhood's representative (root) Intersection, and the value for each entry is the set of all Intersections in that neighborhood that fall within the allowed range of the number of connected roads specified by the input.

This method should have a worst case time complexity of $O(|R| + |I|)$. If either bound is negative or invalid, throw an `IllegalArgumentException`.



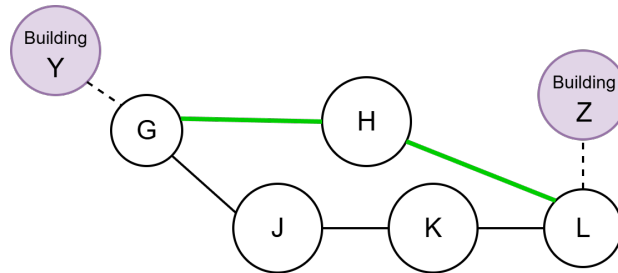
$$\text{Neighborhoods} = \left\{ \begin{array}{l} B: \{B\} \\ F: \{D, E, F\} \end{array} \right\}$$

Calling `getNeighborhoodsByConnections(2,3)` returns the Map above, with the *B* and *F* Intersections serving as the roots. Since only *B* has between 2 and 3 adjacent roads, it is the only Intersection in that entry's set. *D*, *E*, and *F*, meanwhile, all have 2 adjacent roads, so they are all included in that set.

7. getMinBetween

Dr. HB works part time as an EMT driver. She wants to travel on as few roads as possible, to avoid red-light Intersections, when driving the ambulance in an emergency. This method returns the minimum number of roads that must be crossed in order to travel between two Buildings, in $O(|R| + |I|)$ time.

If any of the inputs are `null`, the Buildings are not in the graph, or they do not have a closest Intersection, throw an `IllegalArgumentException`. If there is no route *at all* between the two Buildings, return `-1`.



Min Between = 2

In this example, we call `getMinBetween(Y,Z)`. The path with the fewest number of roads is $[G, H, L]$, which traverses 2 roads. Thus, the answer is 2.

8. calculateRoute

Professor Faulkner drives 18-wheeler trucks in his spare time, but hates driving through certain types of Intersections. His phone's GPS app can find a route that takes the shortest amount of time, but does not smartly avoid undesirable Intersections. You will fix this.

Given two Buildings, this method should find the nearest Intersections to each of the two Buildings, and then calculate the shortest route by duration between those two Intersections while minimizing the number of “bad” Intersections.

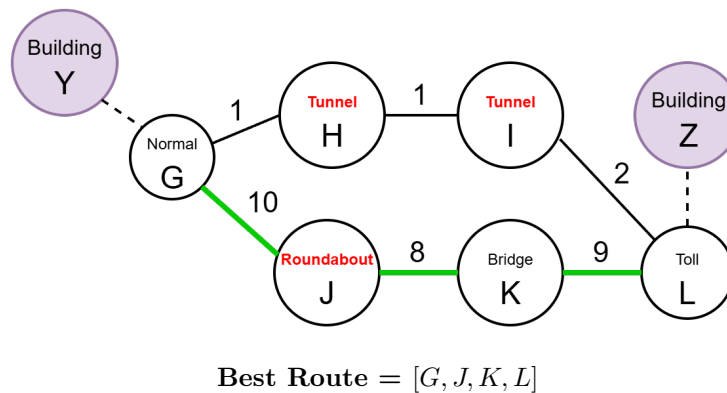
Professor Faulkner prioritizes avoiding the bad Intersection types, even if that means the duration is longer. Ideally, he will take the shortest route with *no* occurrences of the Intersection types to avoid, but if no such route exists, he wants the route with the fewest number of bad Intersections. The total duration of multiple “equally bad” routes should serve as the tiebreaker.

This method must have a worst case time complexity of $O(|R|\log(|R|))$. No space complexity requirement; you can store as many paths as you want in whichever structure you use.

If any of the inputs are `null` or the Buildings are not in the graph, or either Building has no closest Intersection, throw an `IllegalArgumentException`. If there is no route *at all* between the two Intersections, you should return `null`.

Note: This method requires a lot of code. You will need to do more than just call Dijkstra's.

Example: Fewer Bad Intersections

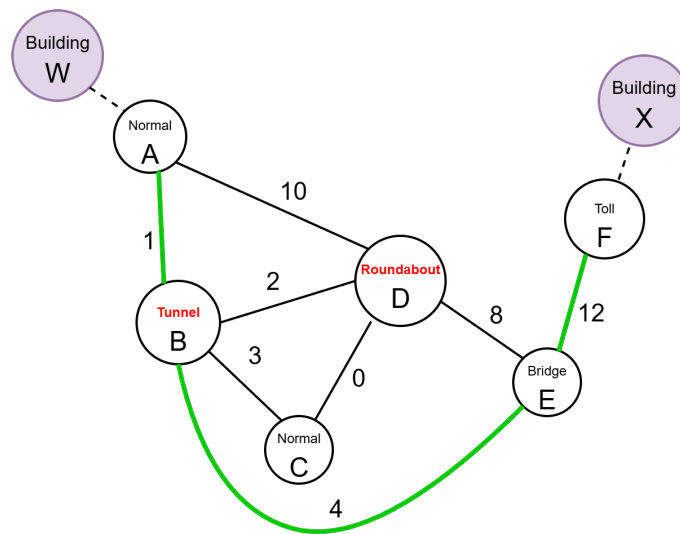


In this example, we call `calculateRoute(Y, Z, {TUNNEL, ROUNDABOUT})`. This means we want to find the shortest route from Building Y to Building Z by traversing through the Road graph, while avoiding Intersections of type TUNNEL and ROUNDABOUT.

First, we find the closest Intersection to each of the two Buildings, which are *G* and *L* respectively. Then, we find the shortest route from *G* to *L* that crosses as few Tunnels and Roundabouts as possible.

The method returns the list of Intersections above, as they define the shortest route that contains the fewest number of bad Intersection types. The best route is highlighted green in the image.

Note that the route $[G, H, I, L]$ has a very short total duration of only 4, but it crosses *two* bad Intersections. The other route, $[G, J, K, L]$ is longer with duration 27 but only crosses one bad Intersection. Thus, **we choose the latter route**, since we care more about minimizing the number of bad Intersections we cross.

Example: Duration Tiebreaker

Best Route = $[A, B, E, F]$

In this second example, we call `calculateRoute(W, X, {TUNNEL, ROUNDABOUT})`.

There are two routes that cross exactly one bad Intersection: $[A, B, E, F]$ and $[A, D, E, F]$. However, since the first route has a shorter duration of 17 versus the second route's duration of 30, we choose the first route. This is an example of using the duration as a tiebreaker.

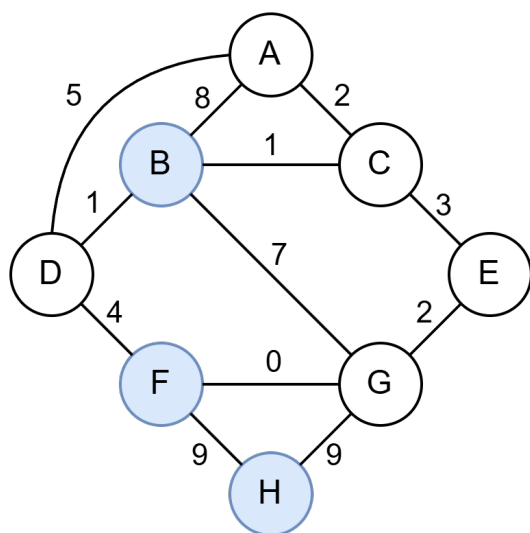
9. getBestPowerSite

Tash Electric, an energy company, is looking for a site in the city to install a new power station. They will locate an existing Building out of a list of proposed candidates, but it must be in the most optimal site for maximum profits!

This method returns the most central Building relative to all other Buildings in the electrical Grid. The *most optimal site* is the one which **minimizes the average distance from all other Buildings**. In other words, it is closest on average to every other Building compared to any other candidate site. The best power site is one of the candidate Buildings passed into the method.

The method must have a worst case time complexity of $O(k(|W|\log(|W|)))$, where k is the number of candidates.

You can return any equally correct result. If the candidate list is `null` or empty, or one of the candidates does not exist in the graph, throw an `IllegalArgumentException`.



Building	B	F	H
A	3	7	16
B	0	5	14
C	1	5	14
D	1	4	13
E	4	2	11
F	5	0	9
G	5	0	9
H	14	9	0
Average	4.125	4.000	10.750

Best Site = F

Candidates = [B, F, H]

In this example, we call `getBestPowerSite([B, F, H])`. The candidates are Buildings *B*, *F*, and *H*. We need to find which of these three candidates minimizes the average distance to all Buildings in the Grid graph.

Find the *shortest* distance from all vertices from each candidate, and the winning candidate is the one that has the lowest average. This is a computationally expensive process. Try doing it by hand!

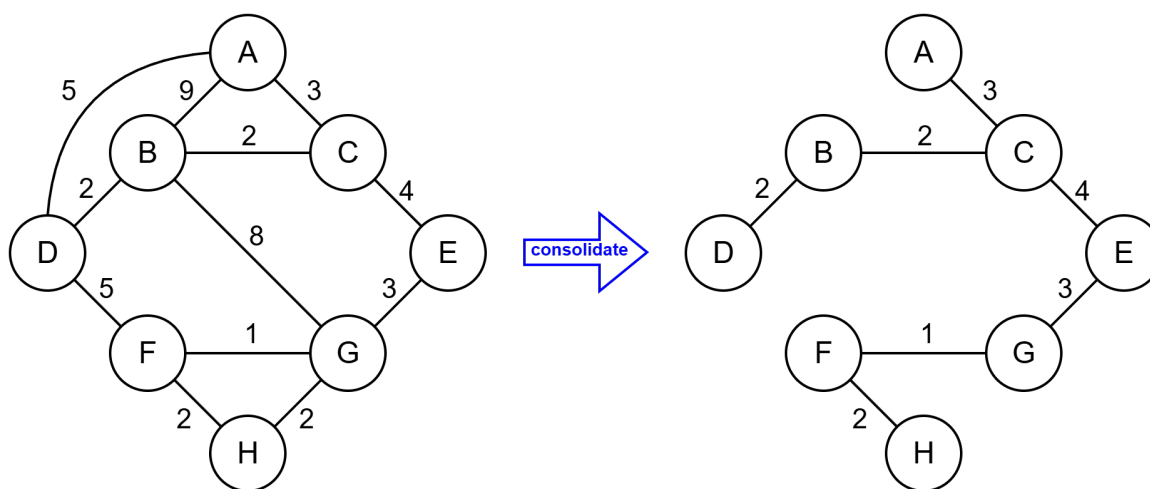
Building *B* has an average of shortest distances of 4.125, whereas Building *F* has 4 and Building *H* has a dismal 10.75. In this case, since 4 is the lowest of the averages, Building *F* is in the most central location out of the three candidates, so the method returns *F*.

10. consolidateGrid

Tash Electric is on a cost-saving mission under its new CEO, Jack. To save on excess wire usage, Jack wants to reclaim all unnecessary wires used to connect Buildings throughout the city. This method modifies the electric Grid graph to remove all unnecessary wiring, thus consolidating the Grid to have the bare minimum number and length of wires.

All Buildings must still be connected to the grid, and the sum of the lengths of all wires should be as low as possible.

Return the fraction of wire that was saved and is no longer in the Grid graph. If the Grid is already perfectly consolidated, you can leave it in its current state. The method should run in $O(|W|\log(|W|))$ time.



Fraction Removed ≈ 0.6304347

In this example, we call `consolidateGrid()` while the Grid graph is in the state on the left, with 8 Buildings and 12 wires (edges). After consolidating the Grid, we remove 5 edges with combined length 29 and are left with a minimal Grid graph that connects all Buildings with the shortest amount of wire. The original total wire length was 46, so the fraction removed is $29 \div 46 \approx 0.6304$ (but don't round).

Grading

Here is the grading breakdown for this project. There are various deductions not listed that are incurred when breaking the rules listed in this document.

Part	Component	
1	Mutable Graph and Disjoint Set	15%
2	Graph Algorithms	10%
3	WaddleWorks	25%
All	Demo	50%

Autograder

The autograder for this course will give you some helpful information, but it will be limited. Every time you submit, you will receive info about any compiler errors, Checkstyle deductions, and general formatting issues. The **first three times per day** you submit, you will see your per-method score. You will not see the number of failed tests or any information about the tests themselves.

Note that while this score is generally representative, **it is not final**. We go over each submission manually, and it is possible for autograder points earned to be deducted in certain cases, including, but not limited to, attempting to brute-force test cases. Additionally, we will manually grade efficiency, so that score is not included in the autograder score. Finally, **we reserve the right to update our test cases** should any defects be discovered, which may result in slight changes to your autograder score.

Checkstyle

Your code must comply with all of the Checkstyle requirements. You will lose a half-point for each Checkstyle issue, capped at -5.0 points. Gradescope will tell you if there are Checkstyle issues in your submission, and you can also manually run Checkstyle by downloading it from Files in Canvas.

Collaboration Policy

Every student is expected to read, understand, and abide by the [Georgia Tech Academic Honor Code](#).

When working on assignments, you **may not** directly copy code from any source (other than your own past submissions). You are welcome to collaborate with peers and consult external resources, but you **must** personally write all of the code you submit. **You must list, at the top of each file in your submission, every student with whom you collaborated and every resource you consulted while completing the assignment.**

You may not directly share any files containing assignment code with other students or post your code publicly online. If you wish to store your code online in a personal private repository, you can use [Github Enterprise](#) to do this for free.